SL2DT-08: Automation & GitOps

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 8 of 11 | **Created:** April 2026 | **Last Updated:** 10/14/2026

Overview

Goal of this step: automate the promotion path for dashboards, monitors, workflows, IAM, OpenPipeline, and bucket configs. Migration is only sustainable if post-cutover changes go through the same CI/CD discipline — not via UI clickops that drift out of source control.

Automation applies both to the migration itself (programmatically import 8,000 dashboards faster than clicking) and to ongoing operations (every config change is a PR).

Table of Contents

1. What You'll Produce
2. Tool Selection — Terraform vs Monaco vs the Future Migration Tool
3. Terraform — Bucket, IAM, Settings 2.0 Config
4. Monaco — Bulk Config Promotion
5. CI/CD Pipeline Patterns
6. Automating the Sumo Extract → DT Import Flow
7. Ansible / CloudFormation / Cloud-Specific Handoffs
8. Post-Migration: Change Management
9. Step Exit Criteria
10. References

Prerequisites

Requirement	Details
Audience	Platform engineering, DevOps, SRE
Tools	Git, Terraform ≥1.6, Monaco ≥2.x, Python 3, cloud CLIs (AWS/GCP/Azure)
Dynatrace access	Platform token for platform resources; OAuth client for IAM and Grail bucket definitions (SL2DT-03 §2, SL2DT-07 prerequisites); separate service identities for CI
Prior reading	AUTOM-01 (Configuration automation overview), K8S-07 (GitOps for Dynatrace), `Dynatrace-NewRelic` for migration-tool patterns

1. What You'll Produce

Artifact	Purpose
`terraform/`	All IaC for buckets, IAM, OpenPipeline, Workflows
`monaco/`	Bulk configs (dashboards, monitors, settings)
`.github/workflows/` (or equivalent CI)	Pipelines for validate → plan → apply
`scripts/extract-and-import/`	Sumo → DT migration pipeline scripts
`runbooks/rollback.md`	How to revert a bad config change

2. Tool Selection — Terraform vs Monaco vs the Future Migration Tool

Tool	Use for	Avoid for
Terraform (Dynatrace provider)	Buckets, IAM groups/policies, Settings 2.0 schemas, some Workflows	Bulk dashboards (slow), complex dashboard JSON

Tool	Use for	Avoid for
Monaco	Dashboards, monitors, settings in bulk; promotion across environments	IAM (use Terraform)
Purpose-built migration tool (not yet published)	Sumo extract + translate + import	Ongoing change management
Direct API (scripts)	One-offs, custom orchestration, Document API for notebooks	Anything that needs state tracking
Dynatrace UI	Investigation, exploration — never for config changes post-cutover	Migration execution

Principles

- **One tool per concern.** Don't manage the same asset in both Terraform and Monaco.
- **State reviewable.** Every config change goes through PR with `terraform plan` or `monaco deploy --dry-run` output.
- **Idempotency.** Every pipeline must be safe to re-run.

3. Terraform — Bucket, IAM, Settings 2.0 Config

Provider Setup

```
terraform {
  required_providers {
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~&gt; 1.60"
    }
  }
}

provider "dynatrace" {
  dt_env_url     = var.dt_env_url         # https://&lt;env-
id&gt;.apps.dynatrace.com
  platform_token = var.dt_platform_token  # platform resources (dt0s16)

  # IAM (Account Management) resources need an OAuth client
  client_id     = var.dt_client_id        # dt0s02.…
  client_secret = var.dt_client_secret
  account_id    = var.dt_account_id
}
```

The argument is `platform_token`. A platform token does not cover everything this notebook manages: *"Platform tokens can't be used for IAM (Account Management) or classic resources."* IAM groups, policies and bindings run on the OAuth client, and the `dynatrace_platform_bucket` resource's own documentation asks for an OAuth client as well (SL2DT-03 §2).

> **Sources:** Dynatrace Terraform provider — argument reference (Dynatrace GitHub), dynatrace_platform_bucket (Dynatrace GitHub).

Organizing Modules

```
terraform/
├── modules/
│   ├── bucket/             # Grail buckets
│   ├── iam-group-policy/   # Group + policy + binding
│   ├── openpipeline/       # Pipelines + processors
│   └── workflow/           # Workflow configs
├── envs/
│   ├── prod/
│   │   └── main.tf
│   ├── preprod/
│   └── dev/
└── variables.tf
```

Example — Bucket Module Call

```
module "bucket_prod_logs" {
  source       = "../../modules/bucket"
  name         = "custom_logs_prod"
  display_name = "Production Logs"
  retention    = 90
}

module "iam_prod_readers" {
  source      = "../../modules/iam-group-policy"
  group_name  = "g_prod_readers"
  description = "Read-only prod logs"
  policy_statements = [
    "ALLOW storage:buckets:read WHERE storage:bucket-name =
\"custom_logs_prod\";",
    "ALLOW storage:logs:read WHERE storage:bucket-name =
\"custom_logs_prod\";"
  ]
}
```

State Management

Use a remote backend (S3 + DynamoDB lock, or GCS, or Terraform Cloud):

```
terraform {
  backend "s3" {
    bucket         = "tfstate-dynatrace-migration"
    key            = "prod/dynatrace.tfstate"
    region         = "us-east-1"
    dynamodb_table = "tfstate-lock"
  }
}
```

Never commit `.tfstate` files to git.

4. Monaco — Bulk Config Promotion

Monaco is the right tool for dashboards/monitors in bulk. It excels at multi-tenant promotion (dev → preprod → prod).

Project Structure

```
monaco/
├── manifest.yaml           # Projects + environment groups + auth
├── projects/
│   ├── dashboards/
│   │   ├── config.yaml
│   │   └── payments-api.json   # One JSON template per dashboard
│   ├── monitors/
│   └── openpipeline/
```

manifest.yaml (non-secret)

```
manifestVersion: "1.0"

projects:
  - name: dashboards
    path: projects/dashboards

environmentGroups:
  - name: production
    environments:
      - name: prod
        url:
          type: environment
          value: DT_PROD_URL            # https://&lt;env-
id&gt;.apps.dynatrace.com
        auth:
          token:
            name: DT_PROD_API_TOKEN     # access token — Settings 2.0 and
classic configs
          platformToken:
            name: DT_PROD_PLATFORM_TOKEN  # documents, workflows, buckets,
segments
  - name: preprod
    environments:
      - name: preprod
        ...
```

Every `auth` entry names an environment variable; Monaco reads the secret from it at run time. Dashboards are platform documents, so the environment needs `platformToken` (or an `oAuth` client) as well as the access token — *"Access tokens and platform tokens are not interchangeable."*

Deploying Dashboards

```
monaco deploy monaco/manifest.yaml --group production --dry-run
monaco deploy monaco/manifest.yaml --group production
```

`--group` (`-g`) selects an environment group and `--environment` (`-e`) a single environment, both by **name** from the manifest. The dry-run is a structural check only: *"A dry-run doesn't connect to Dynatrace and can't validate the content of the JSON sent to Dynatrace."*

Template Parameters

Each config names its template and passes parameters into it:

```
configs:
  - id: dashboard_payments_api
    config:
      name: Payments API
      template: payments-api.json
      parameters:
        prod_bucket: "custom_logs_prod"
    type:
      document:
        kind: dashboard
```

The template references `{{ .name }}` and `{{ .prod_bucket }}`. Values that differ per environment are set with overrides — see **AUTOM-03** for the full configuration format.

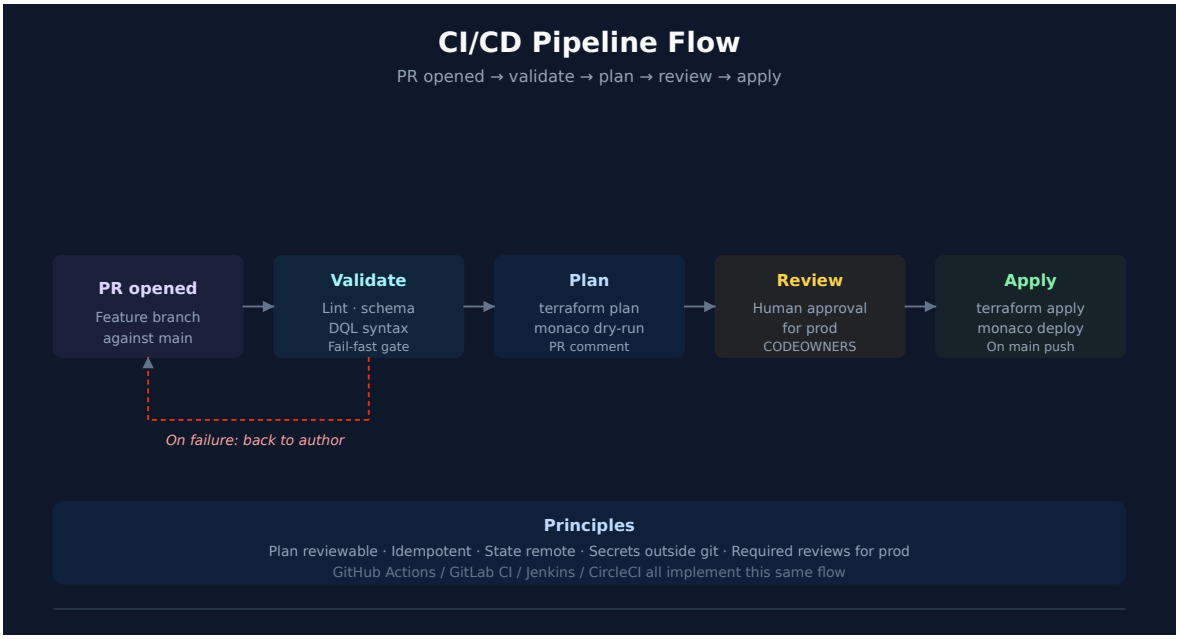
> **Sources:** Monaco commands (DT docs), Manage resources with Monaco (DT docs).

Migration Use Case — Bulk Dashboard Import

If you have 2,000 translated dashboards, iterating via Terraform's `dynatrace_document` is slow (one API call per resource, sequential apply). Monaco batches and deploys in parallel — typical 5–10x faster for bulk work.

5. CI/CD Pipeline Patterns

Typical Pipeline Stages



GitHub Actions Example

```
name: dynatrace-config
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python3 scripts/validate-dql.py terraform/
      - run: monaco deploy monaco/manifest.yaml --dry-run   # structural
check; no tenant contact
      - uses: hashicorp/setup-terraform@v3
      - run: |
          cd terraform/envs/prod
          terraform init -backend=false
          terraform validate

  plan:
    if: github.event_name == 'pull_request'
    needs: validate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: |
          cd terraform/envs/prod
          terraform init
          terraform plan -no-color &gt; plan.txt
      - uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const plan =
fs.readFileSync('terraform/envs/prod/plan.txt','utf8');
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner, repo: context.repo.repo,
              body: '```\n' + plan + '\n```'
            });

  apply:
```

```
    if: github.event_name == 'push'
    needs: validate
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: |
          cd terraform/envs/prod
          terraform init
          terraform apply -auto-approve
      - run: monaco deploy monaco/manifest.yaml --group production
```

Secrets Management

Platform Tokens and API keys live in:

- **GitHub Actions:** repository secrets
- **GitLab CI:** protected variables
- **Jenkins:** Credentials plugin
- **CircleCI:** contexts

Never commit tokens. Every `*.env` file goes in `.gitignore`.

Workload identity federation (SaaS 1.348, staged rollout from 09/22/2026). A CI job or other workload that already has its own identity — a GitHub Actions job, an Azure VM with a managed identity, a Kubernetes pod with a projected service account token — can call the Dynatrace API with a short-lived token from its own identity provider, which Dynatrace maps to a service user through a trust policy. That removes the stored Dynatrace secret altogether. Check that 1.348 has reached your tenant; until then, the CI secret stores above remain the working path.

> **Sources:** What's new in Dynatrace SaaS 1.348 (DT docs) — *"so no Dynatrace client secret or platform token is ever stored alongside your workload, baked into an image, or saved in a CI secret."*; Workload identity federation (DT docs).

6. Automating the Sumo Extract → DT Import Flow

For bulk migration work (Waves 3 and 4), scripts automate the extract + translate + import loop. A purpose-built migration tool (not yet published) would be the destination; until one exists, scripts.

Minimal Python Pipeline

```
#!/usr/bin/env python3
"""Extract Sumo dashboards, translate, import to Dynatrace."""
import os, json, requests, base64
from pathlib import Path

SUMO_AUTH = base64.b64encode(
    f"{os.environ['SUMO_ACCESS_ID']}:{os.environ['SUMO_ACCESS_KEY']}".encode()
).decode()
DT_TOKEN = os.environ['DT_TOKEN']   # dt0s16.XXX
DT_URL   = os.environ['DT_TENANT']
SUMO_REGION = os.environ.get('SUMO_REGION', 'us1')
SUMO_URL = ("https://api.sumologic.com" if SUMO_REGION == "us1"   # us1 has no
region label
            else f"https://api.{SUMO_REGION}.sumologic.com")

def sumo_get(path):
    r = requests.get(f"{SUMO_URL}{path}",
                     headers={"Authorization": f"Basic {SUMO_AUTH}"})
    r.raise_for_status()
    return r.json()

def dt_post(path, body):
    r = requests.post(f"{DT_URL}{path}", json=body,
                      headers={"Authorization": f"Bearer {DT_TOKEN}"})
    r.raise_for_status()
    return r.json()

def sumo_list(path, key):
    """Follow Sumo's `token` / `next` pagination until the list is
exhausted."""
    items, token = [], None
    while True:
        page = sumo_get(f"{path}?limit=100" + (f"&amp;token={token}" if token
else ""))
        items.extend(page[key])
        token = page.get("next")
        if not token:
            return items

def translate_dashboard(sumo_dashboard):
    # Uses sumoql-to-dql skill mapping tables
    # Returns DT notebook/dashboard JSON
    ...

def validate_dashboard(dt_config) -&gt; list[str]:
```

```
    """Local structural preflight on a translated dashboard document.

    Returns a list of problem strings; empty list means the document passed.
    Cheap, offline, and catches the faults a machine translator actually
makes.
    """
    problems = []

    if dt_config.get("version") != 13:
        problems.append(f"version is {dt_config.get('version')!r}, expected
13")

    tiles = dt_config.get("tiles")
    layouts = dt_config.get("layouts")
    if not isinstance(tiles, dict):
        problems.append("tiles is missing or not an object")
    if not isinstance(layouts, dict):
        problems.append("layouts is missing or not an object")
    if not isinstance(tiles, dict) or not isinstance(layouts, dict):
        return problems          # nothing further is checkable

    for tile_id, tile in tiles.items():
        for key in ("type", "query", "visualization"):
            if not tile.get(key):
                problems.append(f"tile {tile_id}: missing {key}")
        if tile_id not in layouts:
            problems.append(f"tile {tile_id}: no matching layouts entry")

    for layout_id in layouts:
        if layout_id not in tiles:
            problems.append(f"layouts {layout_id}: no matching tile")

    return problems


def main():
    out = Path("./translated")
    out.mkdir(exist_ok=True)
    skipped = []
    for dash in sumo_list("/api/v2/dashboards", "dashboards"):
        dt_config =
translate_dashboard(sumo_get(f"/api/v2/dashboards/{dash['id']}"))
        (out / f"{dash['id']}.json").write_text(json.dumps(dt_config,
indent=2))

        problems = validate_dashboard(dt_config)
        if problems:
            skipped.append((dash["id"], dash.get("title"), problems))
```

```
            continue            # never POST a document that failed preflight

        # Dry-run by default; --apply flag pushes to DT
        # dt_post("/platform/document/v1/documents", dt_config)

    for dash_id, title, problems in skipped:
        print(f"SKIPPED {dash_id} ({title}):")
        for p in problems:
            print(f"  - {p}")
    print(f"{len(skipped)} dashboard(s) skipped on preflight failure")

main()
```

Hook `validate_dashboard` into the **PreflightCheck** structure below as a per-document check, so a translation defect is reported in the same shape as a token or scope problem rather than as a stack trace mid-loop.

Preflight Check (before bulk import)

Use a **PreflightCheck** structure for script-level migrations:

```
@dataclass
class PreflightCheck:
    name: str
    status: str              # pass / warn / fail
    scopes_min: list[str]
    scopes_recommended: list[str]
    diagnosis: str
    remediation: str

def preflight():
    checks = []
    # Check token prefix determines auth scheme
    prefix = DT_TOKEN[:7]
    if prefix.startswith("dt0s16"):
        auth_scheme = "Bearer"
    elif prefix.startswith("dt0c01"):
        auth_scheme = "Api-Token"
    else:
        checks.append(PreflightCheck(
            "token_prefix", "fail",
            [], [],
            f"Unrecognized token prefix: {prefix}",
            "Generate a Platform Token (prefix dt0s16) or Classic API Token
(prefix dt0c01)"
        ))
    # ... more checks
    return checks
```

Auth Scheme by Token Prefix

Critical pattern — every HTTP call must pick the right header:

Token Prefix	Scheme
`dt0s16.` (platform token)	`Authorization: Bearer <token>`
`dt0s02.` (OAuth client)	Not sent directly — exchange the client ID and secret for an access token, then `Authorization: Bearer <access-token>`
`dt0c01.` (access token)	`Authorization: Api-Token <token>`

`dt0s01.` is not an OAuth token: it is the account SCIM token — *"This is an API token. It's used as an authorization method: a valid token allows the user to make changes within*

the Dynatrace account through SCIM." The OAuth prefix is `dt0s02`: *"OAuth2 Clients created by users through Account Management to be used with Dynatrace Apps and Account Management API."*

> **Sources:** Dynatrace API authentication (DT docs).

Wrong scheme → `401 Unsupported authorization scheme`, even with every scope.

What the Local Gate Does and Does Not Cover

`validate_dashboard` is a cheap offline filter, not a substitute for the platform's own validation. It catches the structural faults a machine translator actually produces — a wrong `version`, a tile the translator emitted without a `visualization`, a `tiles` entry with no matching `layouts` entry — at zero API cost and before anything reaches a tenant. It cannot tell you whether the document *renders*.

Two limits to be explicit about:

- **A `2xx` from the Documents API does not mean the dashboard renders.** The API confirms the document was accepted for storage. Whether the Dashboards app can draw every tile is a separate question, answered only by opening it.
- **The local gate has no model of the dashboard schema.** It knows the handful of invariants written into it. A tile with a valid-looking but wrong `visualization` value, or a `visualizationSettings` block shaped for a different chart type, passes the local check and fails in the app.

So keep the ladder: local preflight on every document → open a **sample** in the Dashboards app on a non-production tenant → then bulk-apply. The sample is what closes the gap, and on a bulk run it is the only thing that catches a systematic translator defect before it multiplies across the whole wave.

> **Forthcoming/rolling out (SaaS 1.344): a dashboard that fails validation will no longer load.** Published 07/27/2026, staged tenant rollout from 07/29/2026 — verify whether it has reached your tenant. Before 1.344 a failing dashboard still loaded and surfaced validation warnings; after, it does not load at all, and Dynatrace names **API- and AI-authored** dashboards as the most affected population — which is exactly what this pipeline emits. This raises the stakes on a bulk loop specifically: a translator defect that used to ship N dashboards with a warning banner now ships N

> dashboards that do not open. Treat the warning state as a grace period, not a supported state. Full gate: **DASH-07 §5**.

Keep the dry-run default. `--apply` should be an explicit, deliberate act on a wave that has already passed preflight and had a sample opened.

7. Ansible / CloudFormation / Cloud-Specific Handoffs

Many customers run existing automation for infrastructure. Integrate rather than replace.

Ansible (host-based automation)

OneAgent installation via Ansible playbook:

```
- name: Install Dynatrace OneAgent
  hosts: all
  become: true
  tasks:
    - name: Download OneAgent installer
      get_url:
        url: "{{ dt_installer_url }}"
        dest: /tmp/oneagent.sh
        headers:
          Authorization: "Api-Token {{ dt_pass_token }}"
    - name: Install OneAgent
      shell: /bin/sh /tmp/oneagent.sh --set-host-group="{{ dt_host_group }}"
      args: { creates:
/opt/dynatrace/oneagent/agent/lib64/liboneagentloader.so }
```

CloudFormation / AWS

Use the Dynatrace AWS Clouds app (native integration) rather than custom lambdas. Clouds app push path is settled practice — no ActiveGate needed for most AWS metrics.

Terraform for AWS role creation:

```
resource "aws_iam_role" "dynatrace_monitoring" {
  name = "DynatraceMonitoringRole"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { AWS = "arn:aws:iam::509560245411:role/dynatrace-
monitoring-role" }
      Action    = "sts:AssumeRole"
      Condition = { StringEquals = { "sts:ExternalId" = var.external_id } }
    }]
  })
}
```

Azure / GCP

Native cloud integrations via Dynatrace tenant UI or API. Avoid custom exporters unless absolutely needed.

Decision Matrix

Source Platform	Ingest Path
On-prem Linux/Windows hosts	OneAgent via Ansible
AWS EC2 + services	OneAgent (from host) + AWS Clouds app (for services)
AWS Lambda	Dynatrace Lambda extension
Azure VM + services	OneAgent + Azure integration
GCP GCE + services	OneAgent + GCP integration
Kubernetes	DynaKube operator
CloudWatch / EventBridge	AWS Clouds app

8. Post-Migration: Change Management

Once Sumo is decommissioned (SL2DT-09), every Dynatrace config change must go through the same pipeline.

The New Baseline

- All monitors, dashboards, buckets, IAM, OpenPipeline configs in git
- Every change is a PR
- No UI clickops for production changes (UI for exploration only)

Enforcement

- CI blocks any PR with invalid DQL, schema errors, or bucket misconfig
- Required reviews: 1 engineer + 1 platform-team approver for prod
- Settings 2.0 schema drift detection: weekly automated diff between tenant state and git state

Drift Detection Workflow

```
# .github/workflows/drift-detect.yml
name: drift-detect
on:
  schedule:
    - cron: "0 6 * * 1"  # Weekly Monday
jobs:
  drift:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: terraform plan -detailed-exitcode   # exit code 2 = drift in
Terraform-managed resources
      - if: failure()
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.create({
              owner: context.repo.owner, repo: context.repo.repo,
              title: 'Dynatrace config drift detected',
              body: 'See attached diff'
            });
```

`terraform plan -detailed-exitcode` is the drift signal here: it compares state with the tenant and exits `2` when they differ. Monaco has no equivalent — it has no `--report` flag, and a dry-run never contacts the tenant. For Monaco-managed configs, schedule a `monaco download` of production into a scratch folder and review it against git; the

downloaded folder layout differs from a hand-built project, so compare per config rather than with a blind recursive diff.

9. Step Exit Criteria

G8 — Automation Ready

- [ ] All buckets + IAM + OpenPipeline managed via Terraform
- [ ] Dashboards + monitors managed via Monaco
- [ ] CI pipeline runs on every PR: validate → plan → apply
- [ ] Secrets managed outside git (CI vault, not `.env` in repo)
- [ ] Preflight-check pattern used in all migration scripts
- [ ] Ansible / CloudFormation integration documented per source platform
- [ ] Drift detection scheduled + alerting on failure
- [ ] Rollback runbook in `runbooks/rollback.md`

Next step: SL2DT-09 — Cutover, Validation & Decommission (parallel run, validation gates, Sumo teardown).

---

10. References

Dynatrace automation and configuration-as-code

- Configuration as code (DT docs)
- Terraform configuration (DT docs)
- Monaco configuration (DT docs)
- Dynatrace Terraform provider (Terraform Registry)
- Monaco (Dynatrace GitHub)
- API authentication (DT docs)
- Platform tokens (DT docs)

Sumo Logic automation (source)

- Sumo Logic Terraform provider (Terraform Registry)

- Sumo Logic API (Sumo Logic docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official Dynatrace documentation and Sumo Logic documentation.